Salah Circle: A Containerized Multi-User Prayer Tracking Application on Google Cloud

Md Rafiul Islam Rafi
Dept. of Computer and Info. Science, CISC-5550 Cloud Computing
Fordham University
Bronx, New York, USA
mrafi2@fordham.edu

***Abstract*—This paper presents Salah Circle, a full-stack web application that helps Muslims track their five daily prayers and keep one another accountable. The system extends a single-user to-do list application (Homework 4) into a multi-user platform with session management, a social progress feed, a 30-day analytics dashboard, real-time prayer times from the Aladhan API, and personalized motivational messages generated by Google Gemini. The application follows a two-tier microservice design: a Flask REST API with eight endpoints backed by a normalized SQLite database, and a Flask frontend that renders Jinja2 templates and calls the API over HTTP. Both services are deployed on a Google Compute Engine virtual machine, containerized with Docker and Docker Compose, and published to Docker Hub. A deployment to Google Kubernetes Engine (GKE) obtained a public LoadBalancer address but failed to pull images because they were built for ARM64 on Apple Silicon while the cluster nodes run AMD64. We describe the architecture, implementation, and deployment, and report the lessons learned, including graceful degradation for external API failures and multi-architecture image builds for cloud deployment.**

***Index Terms*—Cloud Computing, Flask, REST API, Docker, Kubernetes, Google Cloud Platform, Generative AI, Web Applications**

I. Introduction

Islam prescribes five daily prayers (*salah*): Fajr, Dhuhr, Asr, Maghrib, and Isha. Keeping them consistently is a personal discipline, but it is also strengthened by community. Friends who see each other's progress can encourage one another when a prayer is missed. Salah Circle was built around this idea: a shared tracker in which each user logs prayers, sees friends' daily progress, and receives encouragement.

The project extends Homework 4 of this course, a single-user to-do list composed of a Flask API, a Flask frontend, and a SQLite database deployed on Google Cloud. Salah Circle keeps that foundation and expands it in scope and complexity. The main contributions of this work are:

1) a multi-user, two-service Flask application exposing eight REST endpoints over a normalized relational schema;
2) integration of two external services: the Aladhan prayer-times API and the Google Gemini generative AI API, both with fallback behavior on failure;
3) a 30-day analytics module with a heatmap, a 7-day trend, a streak counter, and a completion rate; and
4) a cloud deployment on Google Compute Engine, containerization with Docker Compose, image publication to Docker Hub, and an analysis of a cross-architecture failure encountered on GKE.

The rest of this paper is organized as follows. Section II describes the Homework 4 baseline. Section III presents the system architecture, Section IV the implementation, and Section V the deployment. Section VI reports results, Section VII discusses challenges and limitations, Section VIII outlines future work, and Section IX concludes.

II. Background: The Homework 4 Baseline

Homework 4 used a two-server architecture on Google Cloud. A Flask REST API (`api.py`) ran on a Compute Engine VM on port 5001 and exposed a single resource, `/api/items`, supporting GET, POST, PUT, and DELETE on to-do items stored in SQLite (`todolist.db`). A Flask frontend (`todolist.py`) on port 5002 rendered HTML templates and called the API over HTTP. The frontend was containerized with Docker, pushed to Docker Hub, and deployed to a GKE cluster with `kubectl`; a LoadBalancer service exposed it publicly. The request path was:

$$\text{Browser} \rightarrow \text{GKE (frontend)} \rightarrow \text{VM (API)} \rightarrow \text{SQLite}$$

The application supported adding, completing, deleting, and listing tasks, with no authentication, analytics, external APIs, or AI. Table I summarizes how Salah Circle extends this baseline.

III. System Architecture

Salah Circle separates presentation from data access into two independently deployable services, following the REST architectural style [1]. Fig. 1 shows the components and their interactions.

A. Backend API

The backend (`api.py`) is a Flask application on port 5001. It owns the database, encapsulates all business logic, and returns JSON. It is the only component that talks to SQLite and to Gemini, so the API key and data access are kept out of the presentation tier.

TABLE I
HOMEWORK 4 VERSUS SALAH CIRCLE

Homework 4 (To-Do)	Salah Circle
Single user, no accounts	Multiple users with Flask sessions
One resource (`/api/items`)	Eight REST endpoints
One table	Two tables with a foreign key
No external APIs	Aladhan API and Google Gemini
No analytics	Heatmap, 7-day trend, streak, rate
No social features	Shared daily progress feed
Basic HTML styling	Custom gold-on-navy themed UI
One Docker image	Two images with Docker Compose

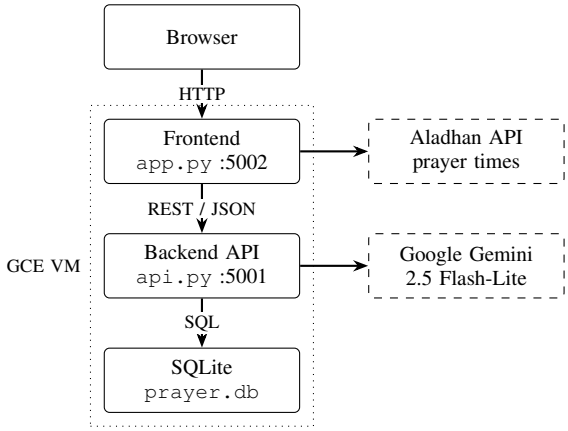


Fig. 1. Salah Circle architecture. Dashed boxes are external services.

B. Frontend Application

The frontend (`app.py`) is a Flask application on port 5002 that renders Jinja2 templates (`index.html`, `analytics.html`). It tracks the current user in a Flask session and calls the backend through the `requests` library. The backend address is read from the `API_URL` environment variable, defaulting to `http://localhost:5001`. This lets the same image run on a VM, under Docker Compose, or in Kubernetes without code changes. The frontend also calls the Aladhan API directly to display prayer times.

C. Data Layer

SQLite was retained from Homework 4. It is a serverless, file-based relational database [2] that suits a small, single-node deployment. The schema is described in Section IV-C.

IV. IMPLEMENTATION

A. REST API Endpoints

Table II lists the eight endpoints. Two design choices are worth noting. First, daily prayer rows are created lazily: when a user's prayers are requested for a date, the API runs `INSERT OR IGNORE` for each of the five prayers, so a day's records exist on first access without a scheduled job. Second, the toggle endpoint flips a prayer's `completed` flag and records or clears `completed_at`, making the operation idempotent from the user's point of view.

TABLE II
BACKEND REST API ENDPOINTS

Method and Path	Purpose
`GET /api/users`	List all users
`POST /api/users`	Create user (or return existing)
`DELETE /api/users/<id>`	Delete user and their prayers
`GET /api/prayers/<id>/today`	Today's five prayers for a user
`POST /api/prayers/<id>/toggle`	Toggle a prayer's completion
`GET /api/social/today`	All users' progress today
`GET /api/analytics/<id>`	Heatmap, 7-day trend, streak
`GET /api/motivation/<id>`	Gemini motivational message

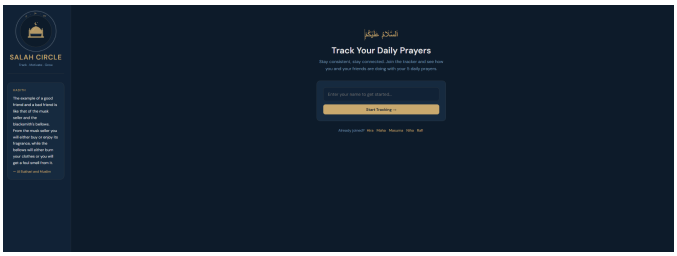


Fig. 2. Landing page where a user joins by name.

B. Frontend Routes and Session Management

The frontend defines seven routes: `/` (dashboard), `/join` (create or retrieve a user by name and start a session), `/switch/<id>`, `/toggle/<prayer>`, `/analytics`, `/logout`, and `/delete-user/<id>`. Because `POST /api/users` returns the existing record when a name is already taken (HTTP 200 instead of 201), joining with an existing name acts as a simple login. Fig. 2 shows the landing page, and Fig. 3 shows the dashboard after joining.

C. Database Schema

The database contains two tables in third normal form:

- `users`(id, name UNIQUE, created_at)
- `prayers`(id, user_id → users.id, prayer_name, prayer_date, completed, completed_at)

A composite UNIQUE constraint on (`user_id`, `prayer_name`, `prayer_date`) guarantees at most one record per user, prayer, and day. This constraint is what makes the lazy `INSERT OR IGNORE` pattern safe. All queries use parameter binding, which protects against SQL injection. The user-deletion endpoint removes a user's prayers before the user row, since SQLite does not enforce foreign keys unless `PRAGMA foreign_keys` is enabled.

D. Prayer Tracking and Social Feed

Each of the five prayers is shown as a card that the user clicks to toggle (Fig. 4). A progress bar shows completed prayers out of five. The social feed (Fig. 5) is served by `GET /api/social/today` and shows every user's status as five colored dots, supporting the project's goal of mutual

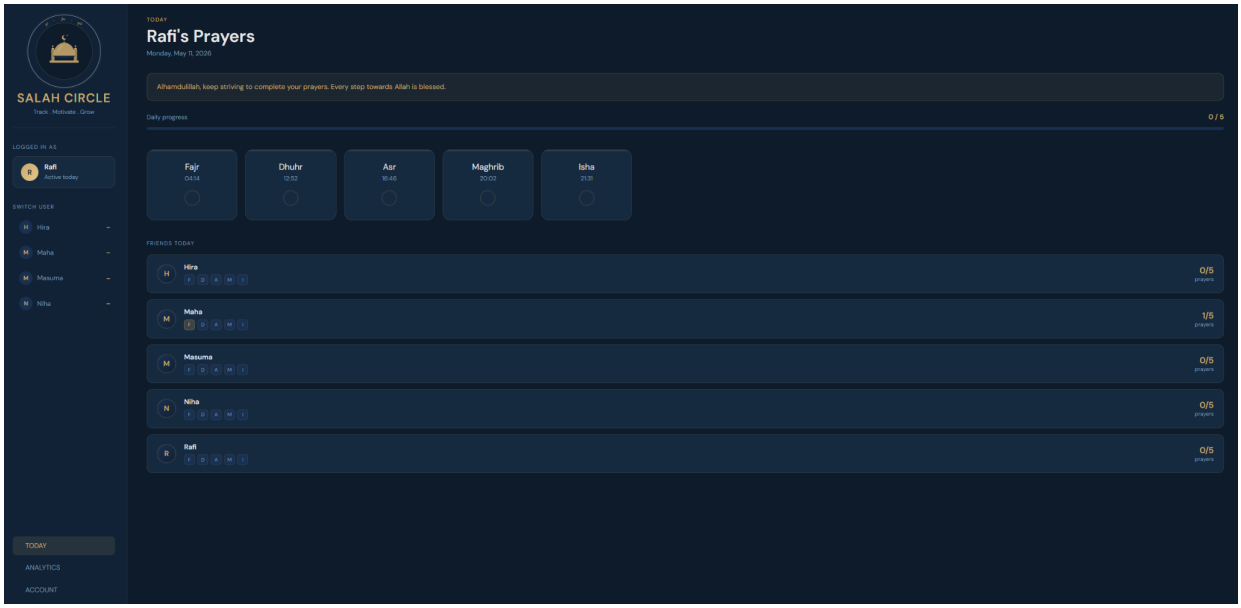


Fig. 3. Main dashboard: prayer cards with times, daily progress, and the social feed.

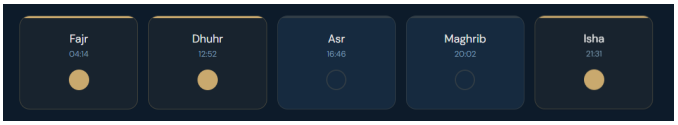


Fig. 4. Prayer cards showing Aladhan prayer times and completion state.

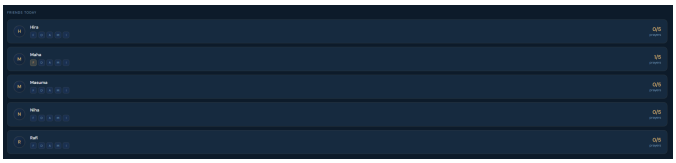


Fig. 5. Social feed showing each user's progress for the day.

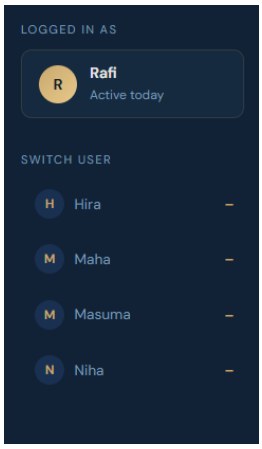


Fig. 6. Sidebar for switching between users.

accountability. A sidebar lets users switch between accounts or delete one (Fig. 6).

E. Analytics

`GET /api/analytics/<id>` returns three structures. The *heatmap* gives the completion state of each prayer for each of the last 30 days. The *7-day trend* gives the number of prayers completed per day. The *streak* counts consecutive full days walking backward from today, up to 365 days:

$$S = \max\{\, k : c(t-i) = 5 \quad \forall\, i \in \{0, \ldots, k-1\}\,\} \tag{1}$$

where $c(d)$ is the number of prayers completed on day d and t is today. The template computes the completion rate from the heatmap:

$$R = \frac{C_{30}}{5 \times 30} \times 100\% \tag{2}$$

where C_{30} is the number of completed prayers in the last 30 days. Fig. 7 shows the analytics page.

F. External API Integration

1) Aladhan Prayer Times: The frontend requests the day's timings from the Aladhan API [3]:

```
GET /v1/timingsByCity/{DD-MM-YYYY}
?city=New York&country=US&method=2
```

Method 2 selects the Islamic Society of North America (ISNA) calculation convention. The call uses a 5-second timeout; on any failure, the cards show `--:--` instead of failing the page.

2) Google Gemini Motivation: The backend uses the `google-generativeai` library with the `gemini-2.5-flash-lite` model [4]. For each dashboard load, it builds a prompt listing the user's completed and missed prayers and asks for a warm, encouraging message of at most two sentences in natural Islamic phrasing. If the call fails, for example because the quota is exhausted or the model is unavailable, the API returns a fixed fallback message. The

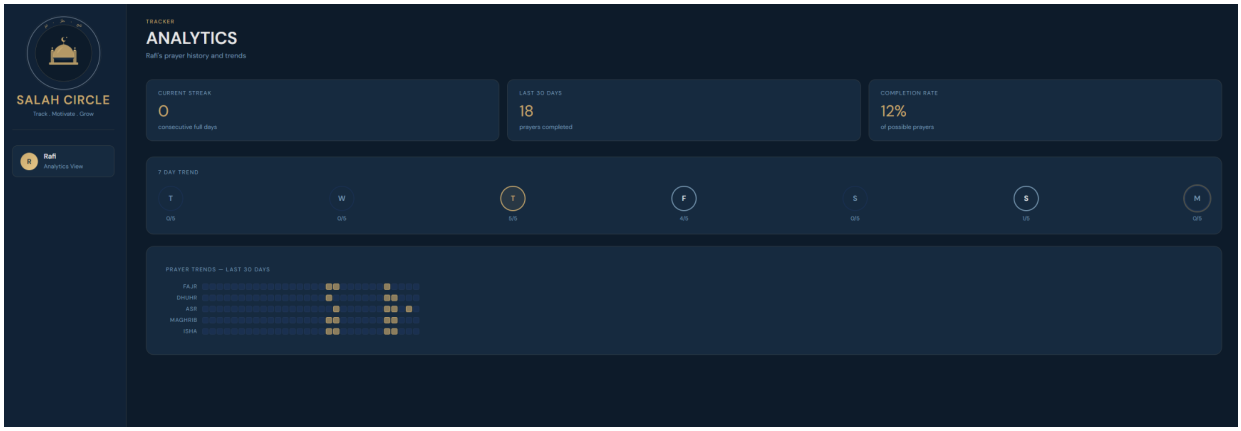


Fig. 7. Analytics page: streak, 30-day total, completion rate, 7-day trend, and 30-day heatmap.



Fig. 8. Motivation banner and daily progress bar.



Fig. 9. Compute Engine instances: the application VM and three GKE nodes.

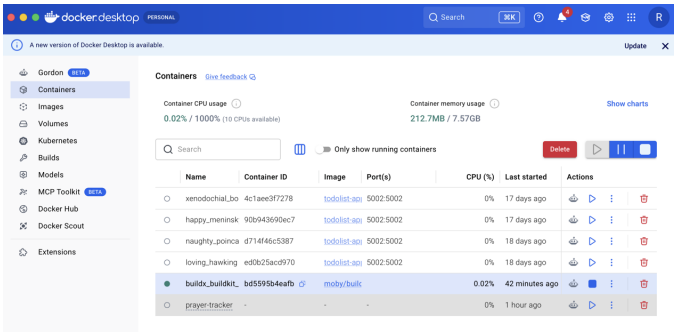


Fig. 10. Containers running locally in Docker Desktop.

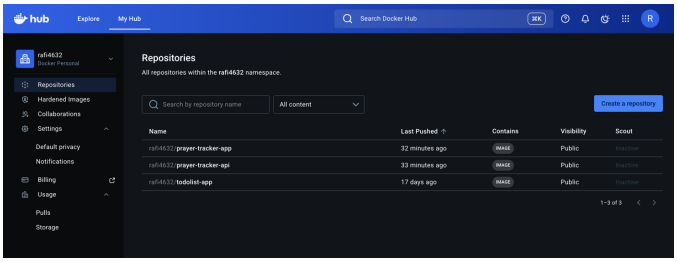


Fig. 11. Published repositories on Docker Hub.

frontend adds a second fallback with an 8-second timeout, so the AI feature never breaks the dashboard (Fig. 8).

V. Deployment

A. Google Compute Engine

Both services run as Python processes on a Compute Engine VM (`homework4-api-server`, zone `us-central1-f`), with firewall rules opening ports 5001 and 5002. This is the application's primary live deployment. Fig. 9 shows the VM alongside the three GKE node VMs.

B. Docker and Docker Compose

Each service has its own Dockerfile (`Dockerfile.api` and `Dockerfile.app`) based on a Python image [5]. A `docker-compose.yml` file runs both containers on a shared network. The frontend reaches the backend by service name (`API_URL=http://api:5001`) via Docker's internal DNS, and the database file is bind-mounted (`./prayer.db:/app/prayer.db`) so data survives container restarts. The stack was built and verified locally with `docker-compose up --build` at `http://localhost:5002` (Fig. 10).

C. Docker Hub

Both images were pushed to Docker Hub as `rafi4632/prayer-tracker-api:latest` and `rafi4632/prayer-tracker-app:latest` (Fig. 11).

D. Google Kubernetes Engine

The Homework 4 cluster (`todo-cluster`) was reused for orchestration [6]. Deployment and Service manifests were written for both services: the API as an internal `ClusterIP` service on port 5001, and the frontend as a `LoadBalancer` service, which received the external IP 34.45.90.149. However, both pods remained in `ImagePullBackOff` (Fig. 12). The images were built on an Apple Silicon (ARM64) Mac, while the GKE nodes run Linux/AMD64. The manifest lists contained no AMD64 variant (only the ARM64 image and build attestations), so the kubelet failed with *"no match for platform in manifest."* The Homework 4 `todolist` deployment on the same cluster continued running normally, which isolates the fault to the image architecture rather than the cluster

```
mrafi2@cloudshell:~ (cischw4)$ kubectl get pods
NAME                                  READY   STATUS             RESTARTS   AGE
prayer-tracker-api-5f6ffd7949-t2bnh   0/1     ImagePullBackOff   0          7m38s
prayer-tracker-app-58768f49c4-wmkvh   0/1     ImagePullBackOff   0          7m37s
todolist-7b5d75f585-b29qc             1/1     Running            0          176m
mrafi2@cloudshell:~ (cischw4)$ kubectl get services
NAME                TYPE           CLUSTER-IP       EXTERNAL-IP     PORT(S)          AGE
kubernetes          ClusterIP      34.118.224.1     <none>          443/TCP          16d
prayer-tracker-api  ClusterIP      34.118.234.43    <none>          5001/TCP         8m7s
prayer-tracker-app  LoadBalancer   34.118.237.137   34.45.90.149    80:31411/TCP     8m6s
todolist            LoadBalancer   34.118.231.94    35.188.204.64   80:32207/TCP     16d
```

Fig. 12. `kubectl` output: Salah Circle pods in `ImagePullBackOff`; the frontend LoadBalancer has an external IP.

configuration.

The standard fix is to build multi-platform images with Docker Buildx [7]:

```
docker buildx build --push \
  --platform linux/amd64,linux/arm64 \
  -t rafi4632/prayer-tracker-app:latest .
```

Building with `--platform linux/amd64` alone would also work. This is left as immediate future work.

VI. Results

The application was evaluated against its functional goals on the live VM deployment.

Functionality. All eight endpoints and seven frontend routes work end to end. Users can join, switch, and be deleted. Prayers toggle correctly, the social feed reflects every user's state, and the analytics page renders all four metrics. Fig. 7 shows a sample user with 18 prayers completed over 30 days, a completion rate of $18/150 = 12\%$ by (2), and a streak of 0 because the most recent day was incomplete; this is consistent with (1).

External services. Aladhan times display on each card (Fig. 4). Gemini generated personalized messages until the free-tier daily quota was reached. After that, the fallback message appeared, as in Fig. 8, and the page continued to work. This confirms the graceful-degradation design.

Deployment. The VM deployment and the local Docker Compose stack both work, and both images are public on Docker Hub. The GKE deployment provisioned networking but not running pods, for the reason analyzed in Section V-D.

VII. Discussion

A. Challenges

Model deprecation. The first Gemini integration used `gemini-pro`, which had been deprecated. Because errors were caught and replaced with a fallback, the failure was silent: the dashboard simply showed the default message. The exception is printed to the server log, and the issue was fixed by switching to `gemini-2.5-flash-lite`. The lesson is that fallbacks protect users but should still be logged and monitored.

API quotas. The free tier allowed about 20 requests per day, and the dashboard calls Gemini on every load. This quickly exhausted the quota during testing.

Cross-architecture images. As described in Section V-D, building on ARM64 and deploying to AMD64 nodes is a common pitfall in cloud-native development and calls for explicit platform targeting.

B. Limitations

Authentication. Identity is a name only, with no password, so any user can act as or delete another user.

Secrets. The Gemini API key and the Flask secret key were hard-coded in the source. They should be revoked and supplied through environment variables or Google Secret Manager.

Persistence. SQLite on a single VM is a single point of failure and cannot be shared safely by multiple Kubernetes replicas.

Time zones. Dates use the server's local date, and prayer times are fixed to New York.

Query efficiency. The analytics endpoint issues one query per prayer per day (150 for the heatmap) and up to 365 queries for the streak. A single grouped query would be more efficient as data grows.

Debug mode. Debug mode is enabled in both services, which is unsafe in production.

VIII. Future Work

Planned improvements are:

1) rebuild the images with Buildx for `linux/amd64` and complete the GKE rollout;
2) move secrets to environment variables or Secret Manager and add password or OAuth authentication;
3) migrate from SQLite to a managed database such as Cloud SQL so that the API can scale horizontally;
4) cache the daily Gemini message per user to reduce API calls and stay within quota;
5) let each user set a location and time zone for accurate prayer times and day boundaries;
6) consolidate the analytics queries and add prayer-time reminder notifications; and
7) add a CI/CD pipeline that builds, tests, and deploys on each commit.

IX. Conclusion

Salah Circle grows the Homework 4 to-do application into a multi-user, socially motivated prayer tracker. It integrates two external APIs, provides a 30-day analytics dashboard, and uses a normalized relational schema with integrity constraints. The two-service design, environment-based configuration, and Docker Compose packaging made the same code runnable on a VM, locally in containers, and, in principle, on Kubernetes. The GKE attempt showed that container portability depends on matching the image architecture to the target nodes. Together with the graceful handling of external API failures, these were the most valuable lessons of the project about building and deploying applications in the cloud.

Live deployment and images:
http://34.42.158.172:5002 (frontend)
http://34.42.158.172:5001 (API)
https://hub.docker.com/r/rafi4632/prayer-tracker-api
https://hub.docker.com/r/rafi4632/prayer-tracker-app

Acknowledgment

The author thanks Professor Zhou Ji for guidance in CISC-5550 Cloud Computing at Fordham University.

REFERENCES

[1] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, CA, USA, 2000.

[2] SQLite Consortium, "About SQLite." [Online]. Available: https://www.sqlite.org/about.html

[3] AlAdhan, "Prayer Times API." [Online]. Available: https://aladhan.com/prayer-times-api

[4] Google, "Gemini API documentation." [Online]. Available: https://ai.google.dev/gemini-api/docs

[5] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, Art. no. 2, 2014.

[6] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Commun. ACM*, vol. 59, no. 5, pp. 50–57, 2016.

[7] Docker Inc., "Multi-platform builds." [Online]. Available: https://docs.docker.com/build/building/multi-platform/